Отчёт по индивидуальному заданию №2

АКОС. Вариант 14 — «Задача об умных цветах»

Горшунов Максим Игоревич, группа 252.
C11, Linux, pthread, libc и системные вызовы.

1. Задача и реализация

Умные цветы высыхают и запрашивают полив. Садовники принимают заявки; при конкуренции одну заявку обслуживает только один садовник. Цветок проходит состояния: нормальное развитие, увядание, полив, восстановление, засыхание или гибель от перелива.

Участник	Работа потока
Каждый цветок	Высыхание, отправка заявки, восстановление
Каждый садовник	Ожидание заявки, выбор FIFO, полив
Главный поток	Контроль завершения, остановка и pthread_join

Потоки асинхронны. Мьютекс защищает общие данные и журнал; семафор каждого садовника будит его при появлении работы. Во время ожидания и полива мьютекс свободен. После выбора заявки садовник делает паузу отклика 1 мс и проверяет её повторно под мьютексом. Если её уже поливает другой садовник, фиксируется конфликт. Победителя определяет планировщик.

Одна стратегия FIFO: сначала старейшая ожидающая заявка. У цветка не более одной активной заявки, у садовника не более одного обслуживаемого цветка. Завершённые заявки не используются снова, погибшие цветы прекращают работу.

Высыхание остановлено при поливе и восстановлении, после полива равно нулю. Полив длится объём работы / скорость с округлением вверх. Попытка нового полива до конца защиты, отсчитываемой от начала предыдущего, убивает цветок. При достижении порога засыхания цветок гибнет.

2. Запуск и завершение

```
make
./build/smart-flowers
./build/smart-flowers --config configs/ordinary.conf
./build/smart-flowers --config configs/conflict.conf
./build/smart-flowers --config configs/drought.conf
./build/smart-flowers --config configs/overwater.conf
./build/smart-flowers --unlimited true --delay-ms 100
```

Без параметров: 3 цветка, 2 садовника, 40 тактов. Справка: `--help`. Настраиваются число участников, скорости, пороги, полив, восстановление, защита и длительность. Файл содержит `key=value`; CLI важнее файла. Один такт длится `delay-ms` миллисекунд, при 0 — 10 мс. Время монотонное, порядок событий между запусками может различаться.

События и итоги записываются в терминал и `events.log` через дескрипторы файлов. Завершение: срок, гибель всех цветов или Ctrl+C. При нуле цветов завершение немедленное; нулевая длительность завершает ограниченный режим. После остановки потоки присоединяются, сохраняются итоги, закрываются семафоры и журнал, освобождается память.

3. Результаты проверки

Данные	Наблюдаемое поведение
ordinary.conf	Завершены 3 полива, цветок жив
conflict.conf	8 поливов завершены, 4 ещё идут, 15 конфликтов
drought.conf	2 цветка засохли, 1 остался жив
overwater.conf	После первого полива новая заявка привела к гибели в t=4

В таблице приведён один запуск каждого сценария; порядок и счётчики могут меняться. `make test`: Linux — 74 проверки, macOS — 72; две проверки `/dev/full` пропущены на macOS. Ошибок нет. Сборки GCC и Clang без предупреждений. ASan/UBSan и отдельные запуски ThreadSanitizer — без диагностик; утечки памяти на macOS не проверялись. Проверены также Ctrl+C и отказ создания потока. Протоколы и журналы: `docs/validation`.

4. Использование AI

ИДЗ-2 основано на коде ИДЗ-1.

1. "помоги составить тесты для моей программы"
2. "провалидируй мой код с точки зрения работоспособности и поправь мелкие баги, если они есть"
3. "Провалидируй, что мой отчет соблюдает все требования ТЗ"
4. "Отрефактори мой код, чтобы он соблюдал стилистические нормы C11 и проходил все линтеры. Основную логику, которую я написал не меняй!"
5. "допиши в отчет и в readme.md команды для запуска моей программы. Укажи там все возможные флоу исполнения с учетом тех тестовых данных, которые ты подготовил ранее!"
6. "оформи мой отчет красиво, добавив какие-то таблицы"
7. "переработой тесты с учетом ИДЗ-2 и написанного мной кода"
8. "допиши отчет с учетом ИДЗ-2"